

ELG5214 Homework 3: Critique and Extension of Decision Transformer

03/31/2026

1 Paper Selection

Decision Transformer: Reinforcement Learning via Sequence Modeling (NeurIPS 2021)

I chose this paper for its relevance to DL/RL, as well as its novelty and reproducibility. I later realized it was included in the in-class presentations, which I unfortunately missed.

2 Paper Summary

2.1 Main Contributions

The paper *Decision Transformer: Reinforcement Learning via Sequence Modeling* proposes a novel framework that reformulates reinforcement learning (RL) as a sequence modeling problem. Instead of relying on traditional RL techniques such as value functions or policy gradients, the authors treat trajectories as sequences and apply a Transformer architecture to predict actions autoregressively.

The core idea is to represent trajectories as sequences of return-to-go, state, and action tuples:

$$(R_1, s_1, a_1, R_2, s_2, a_2, \dots, R_T, s_T, a_T)$$

where the return-to-go represents the cumulative future reward. By conditioning on a desired return, the model generates actions that aim to achieve that reward.

This approach leverages the strengths of Transformer models, including their ability to model long-range dependencies and scale efficiently. The model is trained using supervised learning, where it predicts actions based on past trajectory data.

Empirically, the paper demonstrates that Decision Transformer achieves competitive or superior performance compared to state-of-the-art offline RL methods across multiple benchmarks, including Atari, OpenAI Gym (D4RL), and Key-to-Door tasks.

In short:

- It shows that reinforcement learning can be framed as conditional sequence modeling.
- It removes the need for explicit value estimation, policy improvement steps, and Bellman backups.
- It demonstrates competitive offline RL performance on standard benchmarks such as Atari and OpenAI Gym.

2.2 Literature Gap

Traditional reinforcement learning methods often rely on bootstrapping, which can make training unstable and difficult to scale. Decision Transformer offers a simpler alternative by using the strengths of Transformer-based modeling to learn policies directly from offline trajectory data.

2.3 Methodology

Decision Transformer treats a trajectory as an ordered sequence of tokens. At each timestep, the model receives the return-to-go, the current state, and the previous action. The return-to-go acts as a conditioning signal that tells the model what level of future reward to aim for. In this way, the policy is guided not only by the current state, but also by the desired outcome.

The model processes the last K timesteps of context, which gives a total of $3K$ input tokens. These tokens are embedded and combined with positional embeddings before being passed into a causal Transformer. The output corresponding to each state token is then used to predict the next action. Training is done on offline trajectories using a standard supervised objective, such as cross-entropy for discrete actions or mean-squared error for continuous actions.

This methodology is important because it avoids many of the difficulties of standard RL training. Rather than learning through repeated interaction, value estimation, and bootstrapped targets, the model learns directly from previously collected data using a stable sequence prediction framework

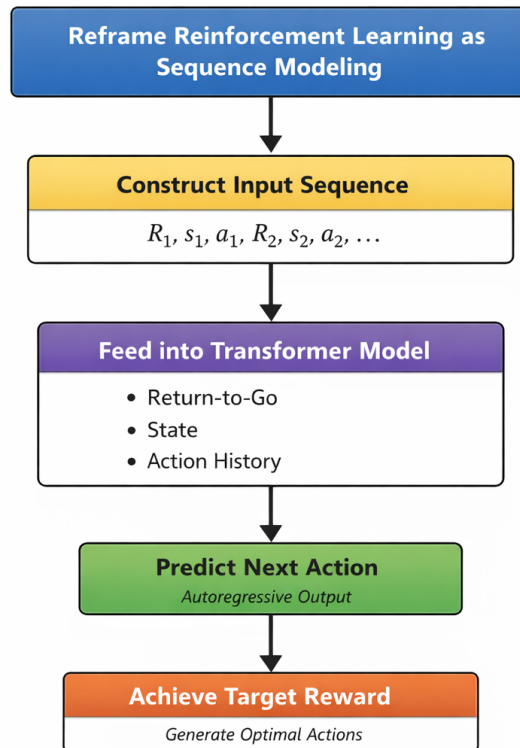


Figure 1: Decision Transformer Model Flowchart

2.4 Key Results

Decision Transformer achieves competitive or better performance than strong offline RL baselines on standard benchmarks such as Atari, OpenAI Gym, and Key-to-Door, even though it uses a much simpler supervised training objective instead of Bellman backups or policy gradients. The results also show that the model is especially effective in settings that require long-term credit assignment and it remains strong when rewards are sparse or delayed, where TD-learning methods can fail. More broadly, the paper shows that sequence modeling is not only a simple replacement for traditional offline RL components, but also a promising general framework for reinforcement learning. At the same time, the authors note that more work is needed to understand the model’s failure modes, the effect of dataset bias, and the risks of conditioning behavior on desired returns. The authors also suggest several directions for future research to further improve the model.

3 Comparison with Related Work

A closely related line of work I know is Conservative Q-Learning, a state-of-the-art offline RL method. CQL learns a value function while applying a pessimistic penalty to avoid overestimating unseen actions.

The main differences between Decision Transformer and CQL are:

- **Learning paradigm:** CQL is value-based and relies on Bellman updates, while Decision Transformer is purely supervised and autoregressive.
- **Stability:** Decision Transformer avoids bootstrapping instability, whereas CQL must carefully regularize value estimates.
- **Modeling approach:** Decision Transformer models full trajectories, capturing long-term dependencies, while CQL focuses on step-wise value estimation.

Decision Transformer fills an important gap by demonstrating that competitive RL performance can be achieved without explicit value learning, using only sequence modeling.

Another closely related work that published at similar time is *Offline Reinforcement Learning as One Big Sequence Modeling Problem*, which proposes the Trajectory Transformer. Like Decision Transformer, it applies Transformer-based sequence modeling to reinforcement learning. However, Decision Transformer uses return conditioning to predict actions directly, while Trajectory Transformer models full trajectories and uses beam search for planning.

Decision Transformer is an important early work that reframes reinforcement learning as a sequence modeling problem. By treating action generation as conditional prediction based on a desired return-to-go, it offers a simpler alternative to traditional RL methods that rely on value functions and bootstrapping.

4 Critique

A key limitation of Decision Transformer is that it is not very good at combining useful parts of different trajectories to create a better new one. It mainly learns to copy action patterns from the data it sees, so its performance is often limited by the quality of the training trajectories. This makes it less effective in cases where the dataset does not contain strong full demonstrations and

better behavior must be built by combining partial successful examples.

Several later works try to improve this weakness of Decision Transformer by adding value guidance or making the sequence model more flexible. *Q-learning Decision Transformer* (QDT) uses Q-learning to relabel return-to-go targets, helping the model prefer higher-value trajectory segments and improving stitching from suboptimal data. *Elastic Decision Transformer* (EDT) addresses the same issue by adjusting the history length, which helps the model avoid getting stuck following a poor past trajectory. More recent work (NeurIPS 2026) such as *Value-Guided Decision Transformer* adds explicit value guidance to improve action selection, especially in harder offline and online settings. Overall, these methods extend the original Decision Transformer by adding mechanisms that make it less like a trajectory imitator and more like a planner.

Another limitation is that Decision Transformer mainly uses *return-to-go* as the condition for action generation. This helps the model target a desired reward, but reward alone does not fully describe how that reward should be achieved. In environments where multiple different strategies can lead to the same return, the model may mix these strategies and produce less consistent actions. This can reduce both controllability and generalization, especially when the target return is ambiguous or outside the training distribution.

5 Proposed Extension

To address the second limitation mentioned above, I propose a *Strategy-Anchor Decision Transformer* (SA-DT). Instead of conditioning only on return-to-go, the model would also condition on a small learned *strategy anchor* that summarizes *how* the reward should be achieved. This anchor could be learned from trajectory patterns such as pace, direction, risk level, or subgoal order, and added as an extra token alongside return-to-go, state, and action. In this way, two trajectories with the same return but different behaviors would no longer look identical to the model. The main idea is that return specifies *how much* reward is desired, while the strategy anchor specifies *what type of trajectory* should be followed. This could improve action consistency, controllability, and generalization in environments where multiple strategies lead to the same reward.

5.1 Hypothesis

A shorter context length is expected to reduce performance because the model uses less trajectory information. A longer context length may improve performance by providing more history, but gains may level off after a certain point.

5.2 Experimental Plan

The experiment tests three context lengths on the Hopper-medium dataset using Decision Transformer:

- Baseline: $K = 20$
- Short context: $K = 5$
- Extended context: $K = 30$

Performance is compared using average return, episode length, and training loss. Success is measured by how these metrics change across the three settings.

6 Reproducibility Discussion

6.1 Code and dataset availability

Like all top journal papers, the paper provides publicly available code and uses standard datasets such as D4RL and Atari replay buffers. Unlike many top journal papers, the code runs reliably and can be reproduced on my own PC with similar results. With three different sizeing of s and clear instructions, this largely improves accessibility and enable makes partial reproduction of the experiments much easier.

6.2 Implementation experience

However, reproducibility is not trivial in practice. The implementation depends on specific software versions, including older versions of Python, PyTorch, and MuJoCo. Setting up these dependencies was time consuming for me.

For the start of this assignment, I was out of town and did not have access to a physical GPU. I first tried using Google Colab, but the Python version was not compatible, which I had partly expected. I then switched to the Morning Star HPC. Since it does not have GitHub connectivity, I uploaded the environment files manually, but the D4RL package kept deleting itself during execution, and I still have not found the cause. As a result, I spent a large amount of time setting up the environment and eventually got everything running on my own PC. While environment setup is expected to take time, the failure of the first two options was beyond my expectations, and running the experiments locally also led to much longer training times.

6.3 Suggestion

Additionally, I would hope the paper includes detailed reporting of hyperparameters and experimental configurations.

7 Optional Implementation

The optional implementation shows a clear overall improvement during training. The evaluation return for the target return of 3600 increases from 1666.41 at iteration 1 to 2288.88 at iteration 10, indicating that the model learns to generate better trajectories over time. At the same time, the training loss decreases from 0.1719 to 0.0613, which suggests that the model is fitting the offline dataset more effectively. Although the return is not strictly monotonic and fluctuates across iterations, the best performance is achieved near the end of training, especially at iterations 9 and 10. This shows that the Decision Transformer improves steadily with training, while maintaining the expected instability of reinforcement learning evaluation.

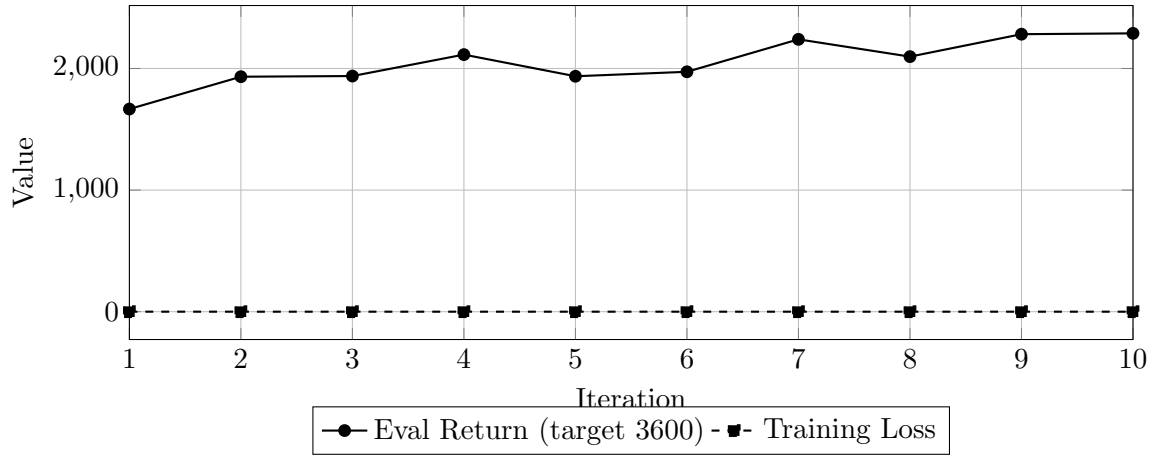


Figure 2: Training trend of Decision Transformer across 10 iterations. Evaluation return generally increases while training loss decreases.

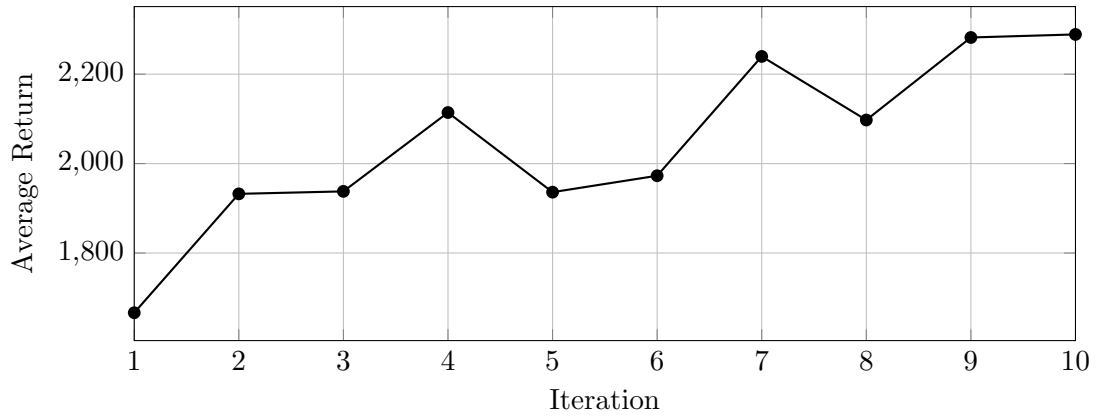


Figure 3: Evaluation return (target 3600) over training iterations.

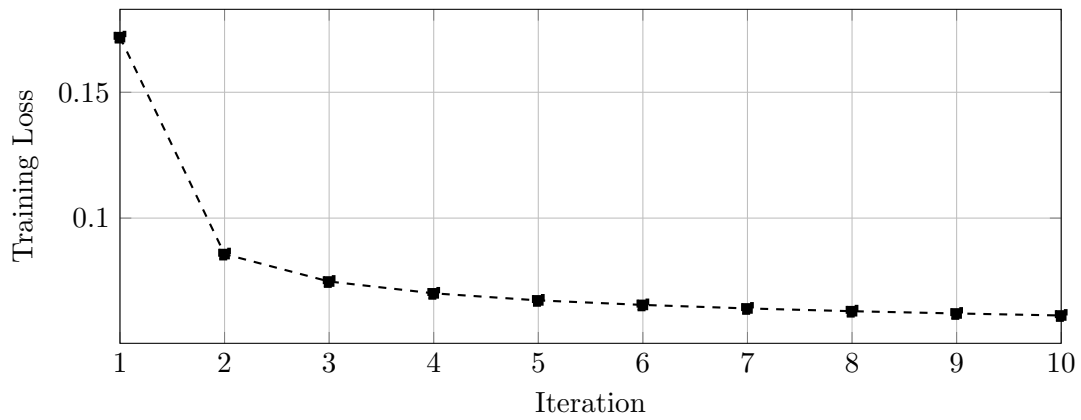


Figure 4: Training loss over training iterations.

8 Reflection on AI Assistance

OpenAI ChatGPT and Google Gemini were used for this homework.

They were used to help with understanding the paper, editing English, suggesting improvements, and refining explanations.